Diffusion Inpainting

Shay Lavi

1 Introduction

Diffusion inpainting is a technique that uses a DDPM-based model[6] to fill in missing parts of an image with new content. Two common methods for inpainting with diffusion models include *preconditioning* and *postconditioning*. *Preconditioned* models learn to inpaint during training. *Postconditioning* approaches do not require training the model, but taking an existing generative model and using it to generate the masked region based on the unmasked region. In this project, we will focus on the latter.
The task at hand is to build a quality inpainting pipeline using stable diffusion[5].
After building the standard stable diffusion inpainting pipeline, we immediately noticed two distinct problems.

- **Background Degradation:** Due to the architecture of stable diffusion, the entire input image must be passed through a Variational Autoencoder (VAE). This step results in the loss of data even in the known region. In some cases, this could result in ineligible text, corrupted faces, and an overall loss of fidelity.
- **Semantic Drift:** During early steps of the diffusion process, the noise applied to the known region is high. This could prevent the model from obtaining much needed context and often results in a mismatching inpaint.

To solve these two issues, we explored several enhancements to the basic algorithm, background reconstruction[1], background copying, mask blurring, a variation of TD-Paint[9], and LatentPaint[3].

2 Methodology

2.1 Stable Diffusion Architecture

In order to explain our approaches to solve the mentioned problems, we must first explain the Stable Diffusion model.

The three major components of the Stable diffusion model[5] are the VAE, U-Net, and scheduler. Most of our methods do not alter the internal function of these components, so we introduce them briefly before detailing our specific solutions.

- **VAE:** Responsible for encoding the image (x) into a lower-dimensional latent-space tensor (z) and decoding it back to pixel-space ($\hat{x}$).
- **U-Net:** Here happens the noise prediction at a given timestep t. Here we find how to denoise z_t to get z_{t-1}.
- **Scheduler:** Defines the denoising schedule and the mathematical algorithm used to iteratively remove noise from the latents during the reverse diffusion process.

2.2 Inpainting Pipelines

We implemented and compared five distinct pipelines to address the issues of background degradation and semantic drift.

2.2.1 Vanilla Pipeline (Baseline)

For the vanilla pipeline we implemented a standard latent-space conditioning approach. We are given an input image x, a mask m[1] and a prompt. The vanilla algorithm is mostly unchanged. The main difference is that before each denoising step, we first apply our condition using the mask:

$$z^* = m \odot z_t^{cond} + (1 - m) \odot z_t^{infr}$$

Where z_t^{cond} is the latent representation of our condition (input image) noised to timestep t, and z_t^{infr} is the latent representation of the inferred image at timestep t. We then denoise z^* to receive z_{t-1}^{infr}. This approach maintains context of the condition while allowing the masked region to evolve during the diffusion process according to the model's prediction. In other words, we "lock" the known region in place while applying the diffusion model to generate the masked region.

Figure 1: Outputs of the vanilla pipeline. Left: input image, Middle: Masked region, Right: Inpainted result.
(Left) Corruption of fine details like facial features and text in the background (Right) Inpainted content lacks semantic alignment with provided scene.

The vanilla pipeline results are, surprisingly, quite good. Although some problems can be clearly seen. Background text could be completely corrupted, along with facial features and other fine details (see Fig. 1). This is due to the lossy nature of the VAE, described earlier. Another problem we get from the standard approach is semantic drift. In some cases, the inpaint does not exactly match the background, or at all.

2.2.2 Background Copy (Naïve approach)

To eliminate background degradation caused by VAE reconstruction errors, this pipeline implements the simplest approach. Here we perform a pixel-level paste of the original background on top of the decoded output, Formally:

$$I_{final} = m \odot x^{cond} + (1 - m) \odot \hat{x}$$

Where $\hat{x}$ is the decoded output of the diffusion process. This guarantees a pixel-perfect background, faithful to the original input.

The simplicity of this approach is nice. But in some cases, it results in distinct outlines along the mask edge (see Fig. 2). This is a direct result of the stable diffusion process. The VAE encodes the input and mask:

$$x \in \mathbb{R}^{3\times512\times512}, E(x) \in \mathbb{R}^{4\times64\times64}$$

Therefore, we cannot effectively isolate masked and unmasked pixel-space regions with perfect precision. Especially if the unmasked region is small and surrounded by masked pixels, or contains high frequency details (Which is also the failure mode of the vanilla with background degradation). In such cases, it is inevitable that the unmasked region will undergo significant alterations which will likely make the pixel-space pasting extremely noticeable.

[1]Commonly, $m = 1$ stands for masked region and $m = 0$ stands for the unmasked (or known) region. In this project we use the opposite convention.

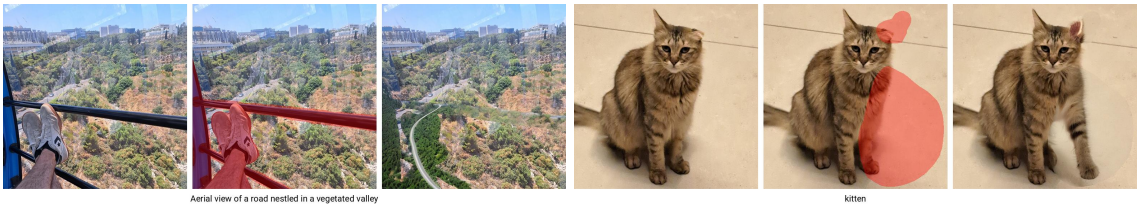

Figure 2: Failure mode of the background copy method. Clear outline artifacts, leftover from the mask

2.2.3 Background Reconstruction (Through Weight Optimization)

Weight optimization[1] refers to optimizing the decoder weights. Specifically, fine-tuning the parameters θ of the VAE decoder D_θ to minimize a reconstruction loss of a single input image x and its latent representation z_0:

$$\theta^* = \underset{\theta}{argmin}||(1-m) \odot D_\theta(z_0) - (1-m) \odot \hat{x}|| + \lambda||m \odot D_\theta(z_0) - m \odot x||$$

Where x is the original input image, $\hat{x}$ is the decoded output of the vanilla pipeline, and λ is a hyperparameter representing the trade-off between background preservation and persistence of the newly generated content. By optimizing the decoder weights in this way, we effectively overfit the VAE on a single image. The objective is to minimize MSE loss between the unmasked area of the vanilla pipeline output and the input image, while at the same time minimizing the MSE loss of the masked regions of the decoded outputs to preserve the generated inpaint.

Figure 3: (Top) Results of lite weight optimization (Bottom) Results of full weight optimization

We implemented two versions of this pipeline. This trial aimed to determine if a smaller parameter set could achieve sufficient background restoration while reducing computational overhead. The fully fine-tuned variant (which is also the one introduced in the paper we read[1]) trained the *entire* VAE decoder. This ensured that we could obtain similar results to those presented in the paper. While this approach is computationally expensive, it provides high capacity to reproduce the background. The light variant included training only the `mid_block` and `conv_out` layers of the decoder.

Both pipelines can reconstruct facial features and text well (Fig. 3). The **light optimization** pipeline occasionally exhibited a "smearing" effect, where high-frequency details in the background weren't corrupted, but completely smoothed over. Conversely, **full optimization** of the decoder led to much superior sharpness and fidelity of the background but introduced somewhat sharp edges around the mask. While these artifacts were significantly less severe than the previous approach of copying the background, they still presented a trade-off between faithfulness to the unmasked region and visual harmony of the two regions.

Implementation details: We followed the training strategy presented in the paper[1]. We used the ADAM optimizer with a learning rate of 0.0001 and $\lambda = 100$ which ensured high priority of background preservation. Every weight optimization consisted of 75 iterations of optimization.

2.2.4 Mask blurring (Occam's razor)

As a computationally efficient alternative to the background reconstruction approach, we explored a simpler technique focused on spatial consistency. Before downsampling the binary mask $m \in \{0,1\}^{512\times512}$ to the latent space dimensions, we apply a Gaussian blur kernel $m_{soft} = m * G_\sigma$. The motivation behind this approach is to mitigate the sharp discontinuities identified in the Background Copy method. By softening the mask edges in pixel space before they are encoded by the VAE, we create a "transition zone" where the diffusion model can naturally blend the generated content with the original background features. By using a non-binary mask, this "transition zone" would contain a mix of signals from the generated region and from the original input signal. This Could ensure

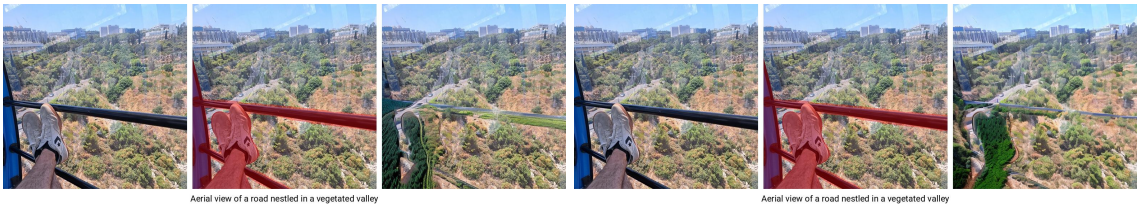

Figure 4: (Left) Result from Copy and Blur pipeline. (Right) Result from Mask Blur pipeline

A problem we noticed in this attempt was that for some examples where the mask tightly envelops a subject (See Fig. 4), the outer parts of the subject are visible in the resulting generation. This outcome may not be desirable for certain use cases.

We implemented Background Copy and Mask Blur as two separate pipelines, but also made a pipeline that utilizes both techniques (`CopyAndBlur`). Mask was blurred with a 13×13 kernel and $\sigma = 10$

2.2.5 TD-Paint

While experimenting with the vanilla pipeline, we discovered a problem that we thought was worth investigating. We recorded the inpainting process (See `README.md`) and found that the "known" region does not provide a useful signal in the first iterations. That is no surprise, of course, because the known region is being noised according to the current timestep t. But it did make us wonder, how could we make the background provide context to the model from the start? Our first idea was to noise the known region using a different, smaller timestep, such as $t^{cond} = \lfloor \frac{t}{5} \rfloor$. It's a very simple idea, and we immediately tried it. The results were very poor; we understood it made complete sense because the model was trained on images where noise levels are spatially uniform across the entire latent map. Altering the image in this way made it completely out of distribution for the model.

After researching, we found a paper that describes this exact problem and a better and more thought out method to solve it[9]. The idea is to modify the U-Net to support spatially variant noise. Unlike our naive attempt, which simply reduced the noise level t in unmasked regions and caused a significant distribution shift. Basically, Instead of relying on a global scalar timestep t, we compute a pixel-level (or more accurately, latent-cell-level) timestep map $\tau \in \mathbb{R}^{1\times64\times64}$. This map τ is used to noise the latent representation z_t dynamically. By assigning lower τ values to the "known" unmasked regions and standard t values to the masked regions, the U-Net receives a high-quality background signal from the first iteration

2.2.6 LatentPaint (But it doesn't work)

To mitigate the semantic drift and boundary artifacts identified in the vanilla pipeline, we explored the "Explicit Propagation" mechanism proposed in LatentPaint [3]. Unlike standard latent replacement, which only provides conditioning at the input level, this method injects the noised condition signal directly into various internal layers of the U-Net. By propagating original background features across multiple architectural scales, the model maintains global context from the earliest timesteps, theoretically eliminating the drift observed in the vanilla pipeline.

We attempted to implement this specialized conditioning loop by modifying the U-Net's feature injection path. However, we encountered significant technical challenges regarding the connection of Explicit

Propagation and Latent Space Conditioning modules. Due to the high complexity of these architectural modifications, the fact that this solution requires (a small amount of) training, and the project's timeframe, we were unable to finalize a stable, working version of this pipeline. Our non-functional implementation is preserved for reference at `pipelines/LatentPaint.py`. Despite our failure of implementation we learned much from reading this paper, specifically about the architecture of the U-Net.

3 Experiments

We used a subset of 250 images from the PIPE dataset[8] to evaluate our pipelines. This data set contains images, masks, and prompts. The masks provided in this dataset are inverted. Therefore, before running evaluations, we must download the source images and masks and then invert them using the provided script. For each sample in this dataset, we take (a) an original unedited image, (b) a mask for a portion of that image, and (c) a prompt describing what the mask is obscuring. All in all, our goal is to measure how well each pipeline restored the source image. The evaluation metrics we used are:

- **Mean Squared Error (MSE):** Because one of the problems we looked to solve was background preservation, it makes sense to measure per-pixel errors using MSE[4].
- **Peak Signal-to-Noise Ratio (PSNR):** Quantifies how much of our output is noise in relation to the ground-truth image. This, too, is particularly useful to measure the background region.
- **Gradient Difference (GDiff):** This is a metric we came up with to measure the perceptual coherence of the masked region and the background region. We use a Sobel filter to calculate derivatives of the input and output images, and then we calculate the difference of the values along the mask edge.
- **Kernel Inception Distance (KID):** Compares distributions of generated and real image datasets. Used to assess the quality of generated images. This metric is somewhat similar to FID[2] but is more stable on smaller datasets, which is perfect for our case.
- **Learned Perceptual Image Patch Similarity (LPIPS):** Measures perceptual similarity of two images using specifically trained model[10]. We presume that a good pipeline will exhibit high perceptual similarity to the original input, mainly in the background and edges of the mask, but also inside the generated region.
- **Structural Similarity Index Measure (SSIM):** Unlike simpler metrics like Mean Squared Error MSE or PSNR, which only look at pixel-level differences, SSIM aims to capture the "structural" changes in an image[7]. For example, MSE will return a good score for blurry images because the average value of pixels is close to the original. SSIM will penalize this behavior.

Each metric was evaluated across four distinct regions[2] to capture local and global performance: The entire image, the masked image (generated region), the unmasked region (background region), and a boundary box surrounding the mask.

3.1 Quantitative Results

The evaluation results mostly align with initial expectations while revealing nuanced performance variations across the tested pipelines (Table 1). Quantitatively, background-copying methods (particularly when refined with mask blurring) demonstrate superior perceptual performance across LPIPS, KID, and SSIM metrics. Weight optimization strategies follow as the second-most effective category, with the fully optimized pipeline consistently outperforming the lite variant in terms of reconstruction fidelity and global structural similarity. Our TD-Paint approach consistently exhibits the worst performance, even worse than the vanilla baseline (which is not much of a surprise)

Our evaluation of background preservation and structural integrity (Table 2) confirms that direct pixel-space methods are the most effective at maintaining background fidelity. Here we also receive an assurance

[2]Except GDiff, which was evaluated on three regions: inner mask edge, outer mask edge, and entire mask edge

that our weight optimization techniques are able to achieve significant reconstruction. For background-copying methods, non-zero MSE values are mostly a byproduct of JPEG compression noise rather than model inaccuracies.

A key finding is the performance of our custom **GDiff** metric. While raw background copying preserves background pixels perfectly, it introduces significant edge discontinuities. Our **CopyAndBlur** approach successfully resolves this, achieving the lowest GDiff score (0.364), indicating visual harmony between regions.

Table 1: Global Perceptual Performance on PIPE Dataset (Entire Image)

Pipeline	**LPIPS** $\downarrow$	**KID** $(\times 10^{-3})$ $\downarrow$	**SSIM** $\uparrow$
Vanilla Baseline	0.150 ± 0.079	-1.93 ± 1.93	0.706 ± 0.137
BackgroundCopy	0.073 ± 0.069	-2.85 ± 2.28	0.926 ± 0.074
MaskBlur	0.140 ± 0.076	-1.78 ± 2.27	0.713 ± 0.136
CopyAndBlur	0.066 ± 0.068	-2.98 ± 2.15	0.934 ± 0.070
BackgroundReconstructionLite	0.177 ± 0.096	-0.24 ± 1.98	0.794 ± 0.108
BackgroundReconstruction	0.115 ± 0.067	-1.62 ± 2.14	0.870 ± 0.072
SimpleTDPaint	0.150 ± 0.078	-1.30 ± 2.59	0.704 ± 0.136

Table 2: Reconstruction Fidelity and Edge Continuity Analysis

Pipeline	**PSNR (Unmasked)** $\uparrow$	**MSE** $(\times 10^{-3})$ **(Unmasked)** $\downarrow$	**GDiff (Edge)** $\downarrow$
Vanilla Baseline	26.0 ± 3.7	3.520 ± 3.004	0.950 ± 0.584
BackgroundCopy	52.1 ± 3.5	0.008 ± 0.007	0.983 ± 1.069
MaskBlur	26.4 ± 3.9	3.272 ± 2.925	0.601 ± 0.281
CopyAndBlur	51.1 ± 4.3	0.012 ± 0.013	0.364 ± 0.118
BackgroundReconstructionLite	31.0 ± 4.3	1.280 ± 1.425	0.821 ± 0.581
BackgroundReconstruction	36.0 ± 3.6	0.355 ± 0.344	0.843 ± 0.986
SimpleTDPaint	25.9 ± 3.5	3.537 ± 2.987	0.989 ± 1.056

3.2 Replication

To replicate our results, follow these steps:

- **Download dataset** `python download_dataset.py` with optional flags `--output_dir` and`--samples_count` to control how many samples to download.
- **Invert masks** `python invert.py --samples_dir <dir>`
- **Run pipelines** `python PipelineTester.py --pipeline <pipeline> --src <samples dir> --dst <destination dir`
- **Run evaluation** `python PipelineTester.py --evaluate --src <original inputs> --dst <generated outputs>`. Notice that here `--dst` flag expects to receive the same directory received when generating, not the inner pipeline directories.

4 Work Report

Every major part of the project was implemented by hand. We mostly used Gemini and Claude to fix minor bugs, generate simple automations, and repetitive classes to save time.

- `PipelineTester.py`: We implemented ourselves almost entirely, save for the evaluation section, which was mostly written using Gemini.
- `VanillaPipeline.py`: Is entirely our work. We first generated a very crude POC and then rewrote the file in its entirety. We optimized the pipeline and generalized it to implement the following pipelines as simply as possible.
- `BackgroundReconstruction.py`: We wrote this pipeline in accordance with the paper that introduced the method[1]
- The rest of the pipelines we wrote on our own as well.
- `Evaluator.py` and `Metric.py` we wrote as a basis for the metric classes.
- `KID.py` and `SSIM.py` were implemented by us. The rest of them were mostly generated with some minor manual tweaks and fixes.
- Scripts for downloading and pre-processing the data used for evaluation were generated almost completely.

We began implementing a full LatentPaint[3] implementation. We were very intrigued by the idea and thought it could solve the semantic drift problem. We couldn't make it work because of time constraints and a misunderstanding of fine details in the article. In the future, we'd like to come back and try to implement this idea again. The LatentPaint pipeline class is available at `pipelines\LatentPaint.py` but doesn't work, which is why we didn't include it in the evaluations.

The reason we only implemented the "simple" version of TD-Paint is because of time constraints. This method requires a significant rewrite of the pipeline and complex handling inside the U-Net component, as well as additional training. We enjoyed reading the article and thought the idea related very closely to the problems we noticed in the vanilla pipeline. That is why we still wanted to mention it and at least showcase our (not-so-good) variant.

5 Future Work

There are a few ideas we had but didn't explore fully (or at all) in this project. Some of them are: testing different training strategies for weight optimization, implementing TD-Paint, finishing the LatentPaint implementation, progressive mask shrinking[1], mask enlarging (which could solve the problem posed by mask blurring), and much more...

References

[1] Omri Avrahami, Ohad Fried, and Dani Lischinski. Blended latent diffusion. *ACM Trans. Graph.*, 42(4), jul 2023.

[2] Mikołaj Bińkowski, Dougal J Sutherland, Michael Arbel, and Arthur Gretton. Demystifying mmd gans. In *International Conference on Learning Representations (ICLR)*, 2018.

[3] Ciprian Corneanu, Raghudeep Gadde, and Aleix M. Martinez. Latentpaint: Image inpainting in latent space with diffusion models. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 4334–4343, January 2024.

[4] Alain Horé and Djemel Ziou. Image quality metrics: Psnr vs. ssim. pages 2366–2369, 2010.

[5] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, June 2022.

[6] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[7] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.

[8] Navve Wasserman, Noam Rotstein, Roy Ganz, and Ron Kimmel. Paint by inpaint: Learning to add image objects by removing them first, 2024.

[9] Jiahui Yu et al. Td-paint: Faster diffusion inpainting through time aware pixel conditioning. *arXiv preprint arXiv:2410.09306*, 2024.

[10] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.